that array like any other variable.

Annotating Functions as Variables

It’s pretty common in JavaScript to pass functions as arguments to other functions or to return functions as the result of a function. And in TypeScript that means we need to be able to specify the types of functions when used as arguments or return values.

This gets a little meta, but the type of a function is based on the type of the arguments and the return value. We’ve seen small examples of this in our React code as we pass handlers up and down the stack. In the following snippet, the `setTicketsToBuy` function expects a number argument and returns nothing:

```
interface VenueHeaderProps {
  seatsInRow: number
  setTicketsToBuy: (n: number) => void
}
```

This syntax contains three quirks to be aware of.

First, the type information of the function is carried by the type annotations and not by the names of the arguments. We could specify the type as `(newCount: number) => void`, and that would be the same declaration.

Second, as far as the type system is concerned, there’s no difference between an optional argument denoted with `?:` and a default argument denoted with, say, `result = 0`. From the type system’s perspective, both of those forms mean that the method being described has an optional number of parameters.

Third, you can use type inference to put the function type signature on either side of an assignment.

We’ve already seen this:

```
let priceOfTicket = function(row: number, accessType: string) : number { }
```

This form is equivalent: